

TokenTrail

Track, price and compare what your coding agents really cost

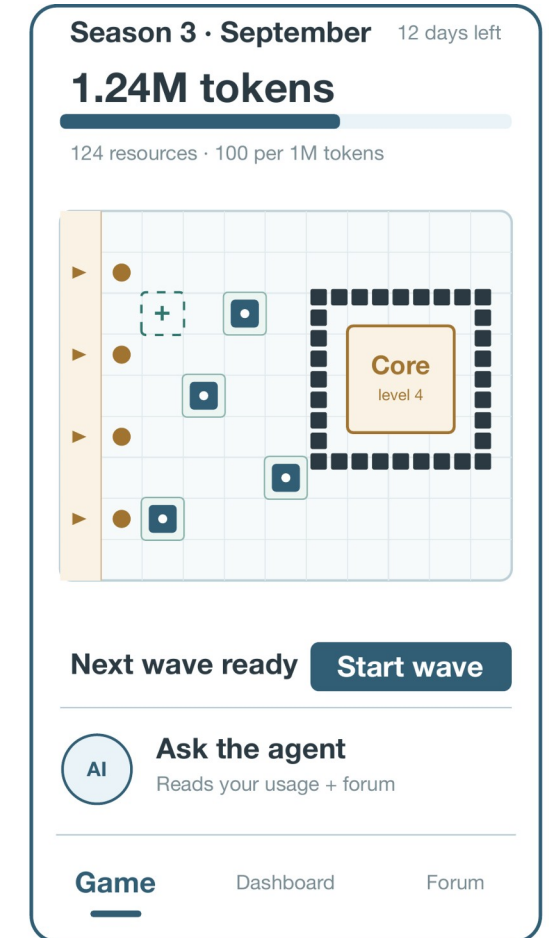
Category: Developer Productivity / API Cost Analytics

Target users: Students and independent developers who use API-key coding agents

Stack: Android · Java/XML · Firebase Auth + Firestore · Room · Java advice service

Group 20 · Zhang Li (24107757) · Wang Tingdong (24107759) · Liu Zongrun (24107745)

Repository: github.com/LiuZongrun7/Mobile_Group_20



The six points the outline slides must cover

Guideline point	What the slides cover	Where
1. Brief app idea description	What TokenTrail is, who it is for, and what is out of scope	Slide 3
2. Discussion of similar / related apps	Codex, ZCode, dsh-context, LiteLLM and dsh-pet	Slide 5
3. Related apps: features and components	What each one covers, and the gap it leaves open	Slide 6
4. Open-source code available to reuse	Libraries, samples and licences we will record	Slide 7
5. Proposed features and uniqueness	Game, analytics, forum and the in-game advice agent	Slides 8-13
6. Approach and work plan to week 15	Alpha, Beta and Final checkpoints with acceptance tests	Slides 14-15

The written outline covers the same six points in the same order; this deck is the visual version of it.

An API-cost companion that makes agent spend visible

TokenTrail is an Android app for people who pay for coding agents with an API key. It prices the usage those agents already logged, turns that usage into a monthly tower-defence season, and answers cost questions with evidence.

Monthly tower-defence game

The home screen. Tokens you have already spent become building resources in a season that resets each month.

API-cost analytics

Prices logged calls from Codex, ZCode and a DeepSeek-side tool using dated official rates, and compares agents.

Community forum

Official model and pricing posts plus user tips; the hot posts feed the in-game advice agent.

And an advice agent inside the game. It reads your logged usage and recent forum posts, then answers questions with the evidence behind them. TokenTrail never runs, routes or controls the coding agents.

Scope: API usage priced at public API rates

Out of scope: ChatGPT and GLM Coding Plan subscriptions, live cross-agent status and system-wide overlays. A response reports tokens for one call, not the agent's whole history, so a call is priced only when a saved rate snapshot covers its date. Every monetary value is an estimate, never an invoiced charge.

Why this is worth building

Each console sees one account

Codex, Z.ai and DeepSeek bill separately, in their own units, with no cross-provider total and no common price version. A user running three agents has three bills and no single monthly figure.

Tokens are not money

A response reports tokens for one call, not the agent's whole history. A call can only be priced when a saved rate snapshot covers its date, so some calls stay Unavailable rather than being guessed at.

Rating your own work is bad input

Asking users to score a task costs them extra effort and produces opinions, not measurements. It also makes the numbers depend on mood instead of on the data.

Spend leaves no trace

A day of agent work leaves nothing visible behind, so usage never turns into progress the user can see and nothing brings them back to the tools.

What TokenTrail does instead

It computes only what the data supports — logged token counts, dated official rates, cache behaviour and duration — and labels every figure Estimated, Partial estimate or Unavailable. It never asks the user to judge how good an answer was.

Five products we studied before writing a line of code

FIRST-PARTY AGENT

Codex

OpenAI's coding agent.
Can run with an OpenAI API key and be billed through the Platform account.

One of the three providers we track.

FIRST-PARTY AGENT

ZCode

Z.ai's coding client. Can use the general Z.ai API endpoint with a key, separate from the Coding Plan.

Tracked through its general API endpoint.

MONITORING PLUGIN

dsh-context

DeepSeek Harness plugin that shows local sessions, token use, cache behaviour, activity and estimated cost.

Its dashboard is our model for the analytics screen.

GATEWAY

LiteLLM

Open-source proxy with routing, spend tracking and budgets — but only for traffic that goes through it.

Teaches the vocabulary of spend tracking and budgets.

COMPANION

dsh-pet

Floating pet driven by DSH session events, with touch and feeding actions. Closest thing to a game.

Its entry point inspires our in-game agent panel.

What the review told us

Every product above is strong inside its own boundary: a first-party console prices one provider, a plugin reads one harness, a gateway sees only proxied traffic. None of them prices three providers from records the user already owns, and only dsh-pet treats usage as something to come back to.

The review also set our boundaries: no proxying, no subscription quotas, and no overlay on another app.

The opportunity

Combining the three ideas is what is new:

- cross-provider costing from logs the user owns;
- a companion-style monthly progression driven by real spend;
- advice that cites its own evidence and admits missing data.

Section 2 of the written outline compares the same five products, with a link to each one.

Component by component: what they cover, what is left open

Product	Relevant capability	Gap addressed by TokenTrail
Codex	Can run with an OpenAI API key, billed through the Platform account.	Estimates only logged Codex API calls, using dated OpenAI prices.
ZCode	Can use the general Z.ai API endpoint with an API key, separate from Coding Plan.	Prices logged general-API calls and excludes plan usage.
dsh-context	Plugin showing local sessions, token use, cache, activity and estimated costs.	Spans API providers; session detail appears only when usable logs are imported.
LiteLLM	Open-source gateway with routing, spend tracking and budgets for proxied calls.	Does not proxy traffic; tracks agent-associated API costs from permitted records.
dsh-pet	Interactive floating pet driven by session events, with touch and feeding actions.	Adds cross-provider, evidence-linked advice, and turns consumption into a visible monthly progression.

Names link to each product's own documentation or repository. The full comparison is section 2 of the written outline.

What we reuse, and what we build ourselves

Layer	Planned tools and rationale
Android interface	Java/XML, Material Components and Navigation; ViewModel/LiveData for state; RecyclerView and MPAndroidChart for trends; a tower-defence game surface and the in-game agent panel.
Network and work	Retrofit/OkHttp for log import and a Java advice service. The service runs one model API and a bounded function-call loop over read-only usage, budget and comparison tools. WorkManager schedules price checks.
Persistence and security	Room plus UID-restricted Firestore; the Java service verifies Firebase ID tokens and holds its model key server-side. Coding-agent passwords and keys are never required.
Open-source reuse	Room with a View guides storage; MPAndroidChart may supply charts; an open-source tower-defence sample may supply the wave and placement loop; dsh-pet inspires the agent's entry point.

Original or properly licensed art only. Every reused file and its licence is recorded in the repository, and this layer is revisited if a dependency changes the plan.

Four contributions, and the rule that keeps them honest

Unified view

Merges authorised API-call logs from Codex, ZCode and a DeepSeek-side tool, removes duplicate records, and shows source and coverage on every figure.

Where the records do not support a session view, the app shows a provider-level estimate instead of inventing one.

Reproducible costs

Java applies dated official rates by provider, model, token class and service tier, and keeps the price version behind each estimate.

New rates apply prospectively; old estimates keep the version they were calculated with.

Reflected progress

Tokens already spent on real work convert into building resources in a monthly tower-defence game — a visible result without asking anyone to spend differently.

The season resets each month, matching the billing cycle AI subscriptions already follow.

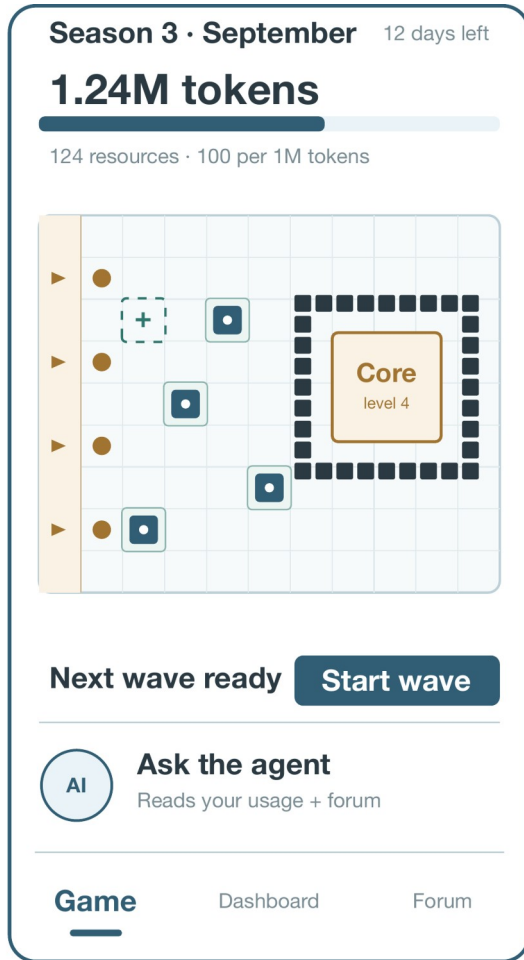
Decision support

Compares agents on computed measures only: logged token counts, dated rates, cache behaviour and duration. The advice agent supplies evidence, and states sample size and missing data.

No task ratings anywhere — the analysis costs the user no extra input, and never judges how good an answer was.

Scope rule: three core parts — the monthly tower-defence game, the API-cost analytics view and the community forum. The advice agent can recommend but cannot execute coding agents or enforce a hard cap.

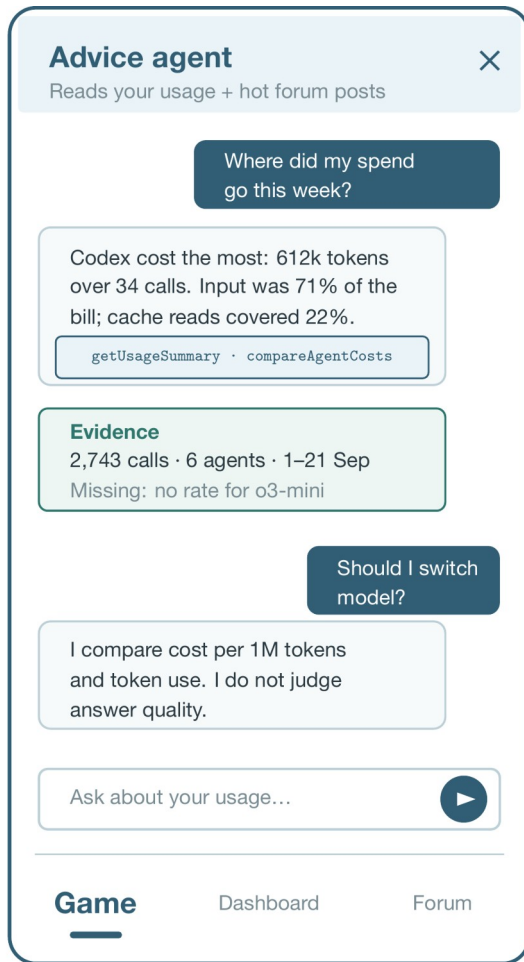
A monthly tower-defence season built from tokens already spent



- **The home screen is the game.** It is played in monthly seasons that match the billing cycle AI subscriptions already follow.
- **Tokens become resources.** Tokens already spent convert into building resources at a fixed rate per million tokens.
- **You plan the space yourself.** Defences and wall segments are placed and upgraded by hand on a build grid around the data-centre core, which expands in one direction.
- **Enemies arrive from one side.** A wave enters from a single edge, so where each tower stands and whether the walls close decide how the wave goes.
- **A wave starts when you start it.** Waves run in real time, so no offline simulation and no scheduled settlement is needed. The season resets each month.

Why it matters. A day of agent work leaves a visible result without asking the user to spend differently, and the resource balance is unreachable without logged usage — so the game cannot be played on invented data.

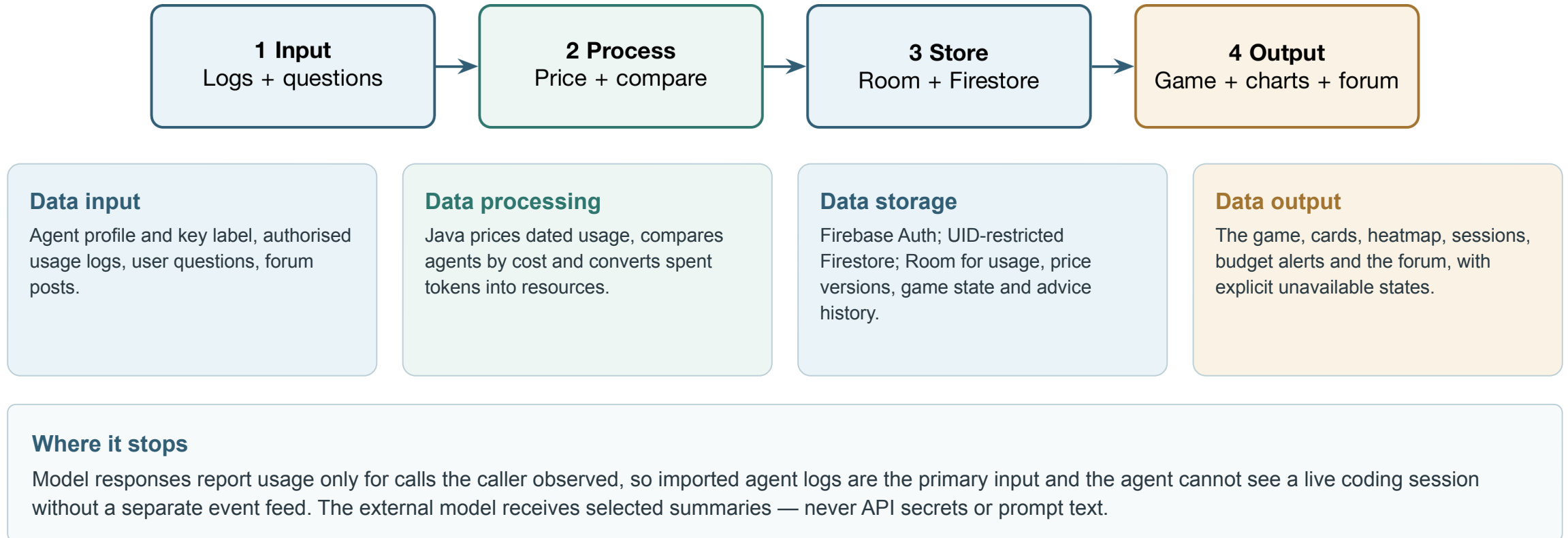
An advice agent that answers from evidence, not opinion



- **It lives inside the game.** An expandable question panel, one tap from the home screen. It reads two sources: your logged usage and recent forum posts.
- **A Java service does the work.** It calls one selected external GPT, GLM or DeepSeek model, which may request read-only tools through function calling. Java executes and validates every call.
- **Four read-only tools.** `getUsageSummary` · `getBudgetStatus` · `compareAgentCosts` · `getForumHighlights`.
- **It admits what it does not know.** Replies state evidence and missing data; no logs means no cost-based answer. Its own tokens are tracked separately from coding-agent usage.
- **Cost, never quality.** It compares agents on cost and token use, and never on how good an answer was.

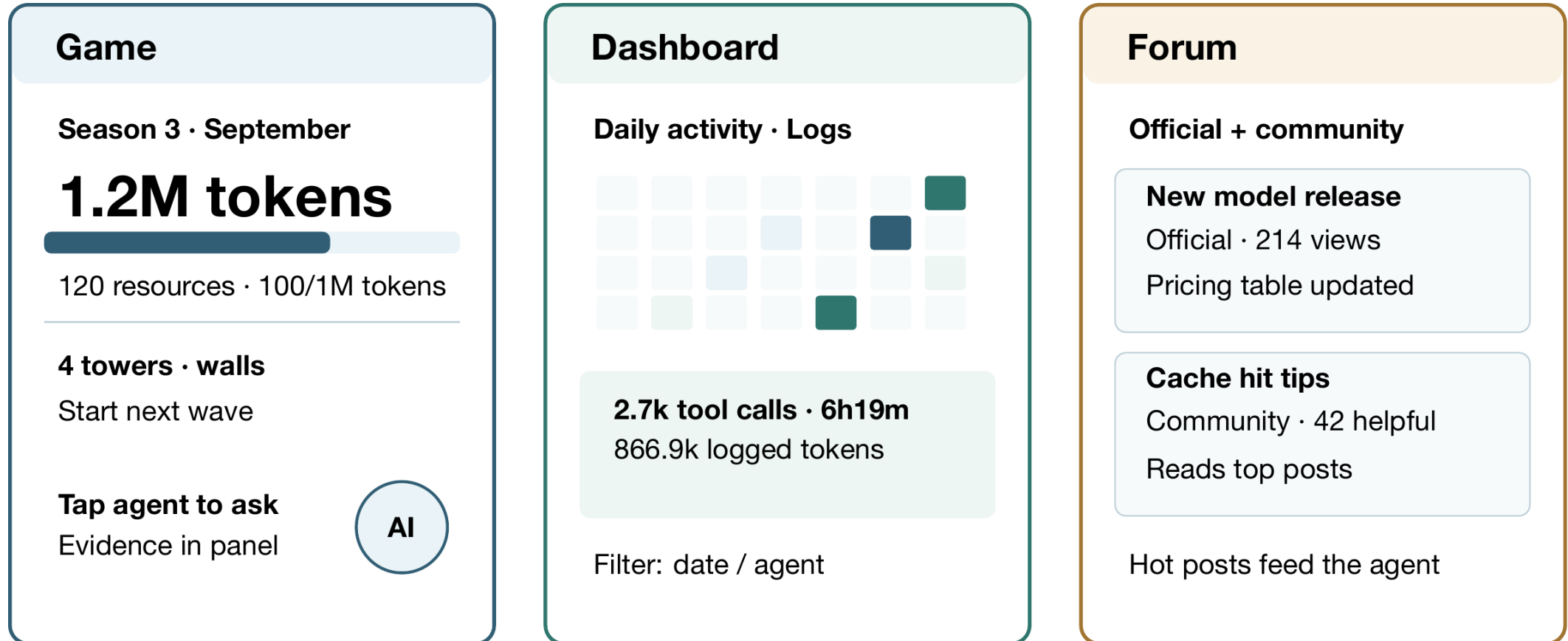
Boundary. The agent can recommend but cannot run, route or interrupt a coding agent, and cannot enforce a hard spending cap.

One path from logs to game, and the four data aspects



The same four aspects (input, processing, storage, output) run through the written outline, the implementation report, the code comments and the final demonstration.

Three screens, with the game as the entry point



The app opens into the game; a bottom bar reaches the forum and the dashboard (Activity, Sessions, Compare, Budget). Inspired by dsh-context and dsh-pet. Log-dependent sections show an empty state when records are unavailable.

Functional requirements mapped to what already exists

Ser	Existing features	Proposed improvements / new features	Remarks
1	Cross-model cost dashboards	Three providers priced from dated official rates, de-duplicated, each figure carrying source and coverage	Improvement
2	Chat-history viewers	Per-call and per-session token-class breakdown, cache read and write split	Improvement
3	Desktop context dashboards	Android client with Firebase Auth and UID-scoped rules	Improvement
4	Provider billing consoles	One cross-provider monthly budget with threshold alerts and a per-session breakdown	Improvement
5	Community forums	Official model and pricing posts plus user tips, ranked for the in-game agent	Improvement
6	Interactive pet with feeding (dsh-pet)	Monthly tower-defence season in which tokens already spent convert into build resources, with one-direction base growth and a wave the user starts	New feature
7	General chat assistants	In-game advice agent with function calling over read-only tools, reading usage records and hot forum posts	New feature

Checkpoints from this week to the final submission

Course weeks	Work	Checkpoint / acceptance
1-3	Confirm API-key agent use cases and UI; set up Firebase Auth and Firestore rules; document usage-log access and pricing dimensions.	Neither test account sees the other's records; token-source limits documented.
4-7	Implement agent profiles, usage-log import, Room/Firestore records, versioned pricing, CNY display and the dashboard.	Each provider's sample calls are priced at the correct dated rate and de-duplicated.
8-9 · Alpha	Deliver a working login → add agent → import log → estimated cost → analytics path; separate real logs from sample data.	Source, description, GitHub link and MP4 per submission guidance.
10-12 · Beta	Add budgets and sessions; build the tower-defence loop (season, token-to-resource conversion, user-started waves) and the agent panel; connect one model to three read-only tools.	The agent answers a usage question with evidence and admits missing data; a wave starts and resolves without blocking the UI.
13-15 · Final	Evaluate advice accuracy and API cost, balance the season, improve accessibility, verify account isolation and document estimation limits.	End-to-end demo, report and video; no agent answer invents usage or controls coding agents.

The team commits to GitHub throughout and adjusts scope based on tested progress.

Who builds what, and how we will know it works

Zhang Li · 24107757

The data side — authentication, usage-log import, Room/Firestore, versioned pricing, budgets and the dashboard.

Wang Tingdong · 24107759

The forum, including its post data and ranking.

Liu Zongrun · 24107745

The tower-defence game, the server-side agent service, its read-only tools and advice evaluation.

Demo path

Account isolation → tracked Codex, ZCode and DeepSeek-side profiles → three provider estimates from dated token logs → a season in which those tokens became build resources → heatmap and budget warning → an agent question such as "Why did cost rise?", answered with a queried summary, evidence and its own estimated API cost.

Validation

Tests cover token accounting, price-version boundaries, missing usage, de-duplication and read-only tool limits, with the Firebase Emulator verifying access. Agent replies are checked against tool data, and the game is checked for a resource balance unreachable without logged usage. Published rates can differ from paid amounts; all figures remain estimates.

Repository: https://github.com/LiuZongrun7/Mobile_Group_20 · UI/UX, accessibility and integration are shared across all three members.